

Complaint & Issue Tracking — Technical Build Spec

Status: Draft technical spec — awaiting Peter's approval before Neo/Q build anything. Not a build yet.

Written by: Oracle

Date: 2026-09-09

Origin: Translates `complaint-tracking-v1-scope.md` (the finalized product design — Asimov- and Mason-reviewed, Peter has resolved every open item in it, including declining two of Mason's recommendations on the record) into a buildable engineering spec. Every product decision in that document is treated as settled and is encoded here faithfully, not re-litigated. This document exists specifically to answer that document's own Section 10 ("Before This Becomes a Real Technical Build Spec") and to design the four mechanisms it named but didn't specify the shape of.

Built from, read in full:

- `projects/hub/email-intake/complaint-tracking-v1-scope.md` — the source design this spec implements.
- `GOVERNANCE.md` (all ten Rules, the Fair Housing Standard) and `CLAUDE.md` — the compliance-build trigger and the agent roster.
- `projects/hub/call-stats/SPEC.md` — the structural and rigor template for this document.
- `projects/hub/property-360/owner-tenant-operational-notes-SPEC.md` and its real, live schema (`supabase/migrations/20260905000000_owner_tenant_operational_notes_schema.sql`) — the closest prior art; reused wherever the shape genuinely matches, diverged from explicitly where it doesn't (see Design Decision 8).
- `projects/hub/owner-tenant-notes/router.js` (full file, both pages) — specifically `proposeAINote`, `checkAIProposedNoteContent`, the role-tier mapping, and the housing-decision-firewall discipline. This spec calls the real, current function signature, not a remembered or assumed one (Design Decision 13).
- `projects/hub/email-intake/lib/privilege-filter.js`, `privilege-keywords.js`, `index.js`, `fair-housing-filter.js`, `shared.js` (full files) — the real hold-check and the real (lighter) email-intake Fair Housing scan.
- `projects/hub/maintenance-history/lib/content-check.js`, `protected-class-terms.js` — the real two-layer Tier A/B content screen the product doc actually asks for (distinct from `fair-housing-filter.js` above — see Design Decision 2).
- `projects/hub/owner-tenant-notes/lib/note-content-check.js` — the sibling-implementation pattern for a table with no `claim_text`, considered and explicitly not followed here (Design Decision 2).
- `projects/hub/email-intake/router.js` and `supabase/migrations/20260905020000_missive_shared_inbox_intake_schema.sql` (full) — the real, currently-built state of email ingestion: messages already land verbatim in `missive_message_intake` with `pipeline_status = 'pending'`; nothing yet reads that table back out or calls the privilege/content filters. This spec designs that missing hookup (see "The Ingestion Pipeline" below) — it does not assume it already exists.

- supabase/migrations/20260813000003_b2_photo_folders.sql and 20260820000000_security_deposit_photo_matches.sql — the real, twice-proven "versioned confidence-threshold config table" pattern (Rule 5), reused for this build's own thresholds (Design Decision 5).
- supabase/migrations/20260626000000_initial_schema.sql, 20260720000002_owners.sql, 20260720000003_foundation.sql, 20260812020000_shared_team_members.sql — real columns for properties, units, tenants, leases, owners, vendors, audit_log, team_members, team_member_tool_roles.
- Rincon's real, live LeadSimple account, queried directly (Design Decision 17) — not just Peter's own description of it.

Where this will live: projects/hub/complaint-tracking/ — a new Hub section (router.js, dashboard/index.html, lib/), mounted into projects/hub/server.js and added as a new tile, gated by team_member_tool_roles with a new tool value 'complaint_tracking'. Same shape every other Hub tool uses (call-stats, security-deposit, maintenance-history, owner-tenant-notes) — no new pattern invented. The product doc's Section 5 already settled that this is a new, dedicated tool, not folded into an existing one.

Neo's Schema Review — 2026-09-09

Verdict up front: sound to build from, after three fixes made directly in this document below — not sound as originally drafted. Read this spec in full against the real prior art it cites (owner-tenant-operational-notes-SPEC.md and its live migration; the real owner-tenant-notes/router.js, privilege-filter.js/privilege-keywords.js, maintenance-history/lib/content-check.js/protected-class-terms.js; the real email-intake/router.js and its missive_message_intake migration; the real b2_match_confidence_config/photo_match_confidence_config precedent; audit_log's real, current CHECK constraints). Every pattern-reuse claim in Design Decisions 1, 2, 5, 6, 11, and 13 checks out exactly against the real code and real schema — checkThread's real shape, checkClaim's real return signature, proposeAINote's real, current required-parameter list (confirmed by reading the live function, not from memory), and the versioned-config/audit_log CHECK values all match what's actually in this codebase today. Three things did not check out, and are fixed in place below rather than just described:

1. A real CHECK-constraint bug that would block the design's own described insert.

complaints_source_requires_fields (Proposed Data Model) required extracted_by IS NOT NULL whenever source = 'email_ai' — but a held Legal/Fair Housing placeholder (Design Decision 1, Ingestion Pipeline step 3) is explicitly never AI-categorized, so it has no classifier version to record, and complaints_held_excludes_ai_fields already forces category/needs_human_call/description to their empty state on a held row. As written, the two constraints together made it impossible to insert a held row through the email pipeline at all. Fixed below by exempting held rows from the extracted_by-required half of that constraint.

2. **A GOVERNANCE.md Rule 5 gap.** Rule 5 requires "every decision must reference the version in effect." Design Decision 5 says this in prose, but the original constraint set left complaint_tracking_config_id nullable with nothing enforcing it — meaning a row that actually used a threshold (duplicate detection runs on every non-held complaint at creation, per Design Decision 10) could be inserted with no record of which version governed it. Added a CHECK requiring complaint_tracking_config_id IS NOT NULL on

every non-held row, matching this same spec's own discipline elsewhere (`blocked_reason`, `flag_requires_category`) of making a compliance requirement a real, structural constraint rather than a comment.

3. A real conflict with this exact codebase's own Rule 9 Housing-Decision Firewall, on the very table this design reads from. `missive_message_intake`'s migration (20260905020000) states, as its single most load-bearing design fact: any association between raw intake content and `properties/units/owners/tenants` "happens only after the existing Stage 0/1/2 filter [the privilege/hold check] has run" and lands in a separate, downstream table — precisely so that unscreened, possibly-privileged correspondence is never linked to a real person's identity record before a human (or the mechanical hold check) has had the chance to pull it out of automated processing. Design Decision 3, as originally drafted, ran the Layer 0 shallow address-match "on every thread unconditionally, **before** the hold check" — inverting that ordering on every single thread, not just the rare held ones. Fixed below: Layer 0 now runs only after `checkThread()` has confirmed a thread is held, solely to populate that placeholder's subject field — the one real, product-required use for it. Non-held threads no longer run Layer 0 at all; the full AI categorization step's own extraction (Ingestion Pipeline step 4, already planned) is the only property/subject match that ever runs against ordinary correspondence, and it runs after the hold check has already cleared the thread, not before. Flagged as its own named item for Asimov, distinct from Design Decision 16's general request for a pass on this document — this is a conflict with a specific, already-governance-cleared architectural decision, not a new judgment call.

Everything else — the six-category CHECK enum, the `blocked_reason/blocked_party/blocked_since` refusal-vs-silence design (Section 10 item 1), the duplicate/merge model, the polymorphic `subject_type/subject_id` pattern, the `is_big_deal` generated column, RLS defaults, and the index set — is consistent with the rest of this schema and does not duplicate anything another table already owns. Detail on each fix is inline at its original location below (Design Decision 3, the Ingestion Pipeline, and the Proposed Data Model), not repeated a third time here.

What This Does

Right now a complaint at Rincon lives as scattered messages in an inbox or a phone call nobody wrote down. This build gives every complaint — however it comes in — one real record with an owner, a status, and a history, while keeping routine noise out of the way: only a "big deal" (six specific categories, or genuine AI uncertainty) gets surfaced prominently. Everything else is still tracked, just not shoved in anyone's face.

How It Works

- 1. Email detection.** A message already sitting in `missive_message_intake` (Rincon's own stored copy, per the Missive connector built this week) gets pulled into this pipeline once its conversation is grouped into a thread.
- 2. The hold check runs first, on every thread, before any AI reads the content for meaning.** A cheap keyword/domain match (`email-intake/lib/privilege-filter.js`, already built) decides whether this is a formal Fair Housing/HUD/CRD complaint or real attorney/legal correspondence. If it trips, a minimal placeholder record is created — date, subject if already metadata-matched, nothing else — and the thread never reaches AI categorization.

3. **Everything else gets AI-categorized:** which of the six big-deal categories (if any) it matches, whether it's genuinely ambiguous ("needs a human call"), which property/tenant/owner/vendor it's about, and whether the conversation's tone is escalating.
4. **A second, independent check** (Maintenance History's real two-layer Fair Housing content screen) tags — never holds — routine correspondence that contains protected-class-adjacent language, most relevant to one-off owner instructions.
5. **The complaint is created** with a real lifecycle (Open → In Progress → Blocked → Resolved), an owner (the Director of Operations for big-deal items), and every stage change logged to this codebase's real audit trail.
6. **A one-off owner instruction outside normal procedure** (Category 6) doesn't just get flagged — it's also proposed as a new entry in the real, already-built Owner & Tenant Operational Notes system, for a human to approve or decline.
7. **Staff can also report an issue by hand**, without ever seeing the tracker itself, through a simple form any Hub login can reach.
8. Peter and the Director of Operations — nobody else — see the full tool, a live count on the Hub home page, and big-deal items on the relevant property's Property 360 page.

What You'll See

- A new **"Complaint Tracking"** tile on the Hub home page (Peter and the DO only), showing a live count of open big-deal items.
- Opening it: big-deal items listed prominently at the top, tagged by category; routine, non-big-deal complaints tucked behind an expand/dropdown, still searchable and filterable, never force-displayed.
- A property's own Property 360 page shows only its big-deal complaints, flat and visible the moment the page loads — no toggle, no click-through.
- A held Fair Housing/legal item shows as a distinctly-labeled placeholder ("Held — Legal/Fair Housing") with just a date and, once handled, a closure note — never AI-generated content.
- Any team member: a plain "Report an issue" form reachable from their own Hub login, with no visibility into the tracker itself or their own submission's status afterward.

What Could Go Wrong

- **The shallow, metadata-only property/subject match that feeds a held placeholder's "subject" field is weak in practice for owners** — `owners.email` is nullable and, per that table's own schema comment, "not available in current AppFolio report." A held Fair Housing complaint from an owner may land with no subject identified at all, which is the correct, safe failure mode (an empty subject, never a guessed one) but worth knowing going in.
- **The AI categorization step is the first place in this codebase that reads full tenant/owner email content to make a triage judgment feeding a human's real workflow** — a materially different, higher-stakes capability than the storage-only Missive connector built this week. It needs its own

shadow-mode period (Design Decision 16), not a assumption that clearing the design doc already covers the code.

- **A wrong duplicate-merge would double-hide a real recurrence as a false "duplicate."** The data model keeps these mechanically distinct (Design Decision 10) and a merge always requires a human confirm — but the underlying "is this a close match" judgment is a real, not-yet-specified algorithm (see Open Items).

Design Decisions

1. The hold check — reusing `privilege-filter.js` directly, not `lib/index.js`'s convenience wrapper

`email-intake/lib/index.js` exports `processThread()`, which bundles the hold check (`privilege-filter.js`'s `checkThread`) together with `fair-housing-filter.js`'s lighter, keyword-only scan into one combined result. **This build does not use** `processThread()`. It imports `checkThread` from `privilege-filter.js` directly for the hold check, and (per Design Decision 2) a different, more capable function for the Fair Housing content tag. Reasoning: `processThread()`'s bundled Fair Housing half is Layer-1-keyword-only — confirmed by reading `fair-housing-filter.js` in full, it calls nothing but `protected-class-terms.js`'s `scanText()`. The product doc is explicit that the content tag should reuse "Maintenance History's real, more advanced two-layer screen," which is a different, more capable module (Design Decision 2). Using `processThread()` wholesale would silently substitute the weaker of the two available Fair Housing checks for the one the product doc actually asked for — a real, easy-to-miss mistake given how similarly the two modules are named.

`checkThread(thread)` expects `{ threadId, legalHoldTag, messages: [{ messageId, from, to, cc, bcc, subject, body, date }] }`. Real data lives in `missive_message_intake` (one row per message, grouped by `missive_conversation_id`) — an adapter maps `from_address` → `from`, `to_addresses/cc_addresses/bcc_addresses` (JSONB) → `to/cc/bcc`, `subject` → `subject`, `body_text` → `body`, `missive_message_id` → `messageId`, `delivered_at` → `date`. `legalHoldTag` **(Layer 3 — a staff "Legal Hold" override) has no real, populated data source today**. No column on `missive_message_intake` carries it, and nothing in the built Missive connector fetches a conversation's Missive-side labels. This adapter defaults `legalHoldTag: false` until that's built — named explicitly as an Open Item, not silently assumed covered.

When `checkThread` returns `held: true`, the whole thread's messages are all marked `pipeline_status = 'processed'` in `missive_message_intake` (holding pulls the entire thread, per the product doc) and one complaints row is inserted with `held_legal_fair_housing = TRUE` and nothing else AI-derived populated (see the table's own CHECK constraint, Section "Proposed Data Model").

2. The Fair Housing content tag — reusing

`maintenance-history/lib/content-check.js`'s `checkClaim()` directly, not a third sibling implementation

Three candidate reuse points exist in this codebase for a "two-layer content check," and picking the wrong one would quietly under-deliver on what the product doc actually asked for:

| Module | Layers | What it's for |

|---|---|---|

| email-intake/lib/fair-housing-filter.js | Layer 1 only (keyword scan via protected-class-terms.js) | A lighter wall-off check for the (still-unconnected) email pipeline. |

| owner-tenant-notes/lib/note-content-check.js | Layer 1 + a **fresh, separate** classification call (classifyManualNote) for the manual path; Layer 1 + caller-supplied self-report for the AI path | Built because operational_notes has no in-flight extraction model to self-report from for manually-typed notes. |

| maintenance-history/lib/content-check.js's checkClaim() | Layer 1 Tier A (immediate flag) + Layer 1 Tier B (a second, narrow, per-instance AI classifier — tier-b-classifier.js — precision-tuned to cut false positives on six ordinary words) + Layer 2 (the extraction model's own self-report) | **This is the "more advanced two-layer screen" the product doc names by name.** |

This build imports checkClaim from maintenance-history/lib/content-check.js **directly** — not a new sibling file. Complaint categorization already has a model reading the full thread content in-flight (Design Decision 6), the same situation maintenance_claims' own extraction is in — so Layer 2 is that same model's self-report (modelFlag/modelCategory), exactly extract-claims.js's convention, with no second AI call needed. Call shape: checkClaim({ claim_text: fullThreadText, modelFlag, modelCategory }), using the returned flagged_protected_class/flagged_category/terms_version/tier_b_results.

One thing this spec does *not* do without Asimov/Mason's separate say-so:

protected-class-terms.js's own header names one narrow, already-approved exception to "never log the matched term" — logging a Tier B triggering term specifically on the maintenance_claims.tier_b_classification audit action. That exception is scoped to that one action, on that one table, by name. This build does **not** extend it to a new complaint_tracking.* audit action by assumption — complaint_tracking's own flag-audit entry logs flagged_category, matched_layer, terms_version, and (if Tier B fired) TIER_B_CLASSIFIER_VERSION only, never the triggering term, unless Asimov explicitly extends the exception to this tool.

matched_layer and tier_b_results are **not** persisted columns on complaints — same restraint operational_notes already applies: only flagged_protected_class/flagged_category are stored; the rest goes only into audit_log.details (Section 9).

3. The shallow, held-only subject match (Layer 0) — corrected by Neo's review, see above

The product doc's held-placeholder description depends on "the subject... if that's already known from earlier, shallower property-matching" — implying a match lighter than full AI content-reading, since a held thread's content is never read for meaning at all. This spec names that mechanism explicitly: a metadata-only match — comparing a held thread's participant addresses (from_address, to_addresses, cc_addresses) against tenants.email, owners.email, and vendors.email (all real, existing, nullable columns). No AI, no body-content read.

Corrected ordering — this is the one place this spec's original draft conflicted with an existing, governance-cleared architectural decision, not a stylistic call. The original draft ran this match "on every thread unconditionally, before the hold check." `missive_message_intake`'s own migration (20260905020000, Rule 9 — Housing-Decision Firewall) states directly that any association between this table's raw content and `properties/units/owners/tenants` "happens only after the existing Stage 0/1/2 filter [the privilege/hold check] has run" — precisely so unscreened, possibly-privileged correspondence is never linked to a real identity record before the mechanical hold check (or a human) has had the chance to pull it out of automated processing. Running Layer 0 before the hold check, on every thread, inverted that ordering for the entire email volume, not just the rare held item.

Fixed: Layer 0 now runs only after `checkThread()` (Design Decision 1) has already returned `held: true`, and only to populate that one placeholder's subject field — the single real use the product doc actually asks for. A thread that is not held never runs Layer 0 at all; its subject/property match comes entirely from the full AI categorization step's own extraction (Ingestion Pipeline step 4), which already resolves property/subject/vendor names against real rows as part of that step, and which only ever runs after the hold check has already cleared the thread. This removes the "fallback/cross-check" role Layer 0 previously played for non-held threads — a real capability given up, but one that was never load-bearing (categorization's own extraction is the actual match for every non-held complaint regardless) and that came at the cost of running a raw-address-to-identity join over the entire ordinary-correspondence volume before the filter had cleared it. See "The Ingestion Pipeline" below for the corrected step order, and Open Items for the specific Asimov callout this correction still needs.

Known weakness, stated plainly, not glossed over: `owners.email`'s own schema comment says it is "not available in current AppFolio report" — a real owner-side data gap that will produce empty-subject held placeholders for a meaningful share of real owner Fair Housing correspondence. This is the correct, safe failure mode (never a guessed subject) but Peter should know the held queue will often show "subject unknown" for owner-originated items specifically.

4. Blocked resolution — "explicit refusal" vs. "inferred from silence" as a real, distinct data-model requirement

The product doc requires these stay visibly distinct "wherever the blocked-resolution tag is shown or reused" — not just in one dashboard view. This is enforced structurally, not by convention:

...

`blocked_reason` TEXT CHECK (`blocked_reason` IS NULL OR `blocked_reason` IN ('explicit_refusal', 'inferred_from_silence'))

`blocked_party` TEXT CHECK (`blocked_party` IS NULL OR `blocked_party` IN ('owner', 'tenant'))

`blocked_since` TIMESTAMPTZ -- when the refusal was received, or when the silence clock started

...

`blocked_reason` is a real, required column (enforced via CHECK — Section "Proposed Data Model") whenever `category = 'blocked_resolution'`, never a free-text note a UI could accidentally omit or blend into one generic "Blocked" badge. Any surface that renders a blocked complaint — the main tool, Property 360, the home-page count's tooltip/detail, any future export or Owner-in-Distress-style rollup — reads this column and

must render it as two visually distinct states (e.g., "Blocked — refused" vs. "Blocked — no response 2+ days"), never collapsed to one badge. This is a hard requirement for whoever (Tron) builds the UI, stated here so it isn't lost between this document and that one.

5. Two clocks, two real thresholds, one versioned config table (GOVERNANCE.md Rule 5)

The product doc names two genuinely different clocks doing two different jobs (Section 4): the 2-day silence window is a **classification** trigger (decides whether something counts as `blocked_resolution` at all); the aging clock is an **escalation** trigger (measures how long an already-confirmed big-deal item has sat before the DO is nudged). Both are real "decision thresholds that affect people" under Rule 5 and must be versioned, never hardcoded — following the exact pattern this codebase has now used twice (`b2_match_confidence_config`, `photo_match_confidence_config`): one row per version, `is_active` boolean, a partial unique index enforcing at most one active row, never UPDATE a threshold in place — insert a new version and deactivate the old.

```
```sql

CREATE TABLE IF NOT EXISTS complaint_tracking_config (

id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

version INTEGER NOT NULL,

blocked_resolution_silence_days INTEGER NOT NULL CHECK (blocked_resolution_silence_days > 0),

big_deal_aging_clock_hours INTEGER NOT NULL CHECK (big_deal_aging_clock_hours > 0),

duplicate_window_days INTEGER NOT NULL CHECK (duplicate_window_days > 0),

is_active BOOLEAN NOT NULL DEFAULT TRUE,

set_by TEXT NOT NULL,

set_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),

notes TEXT,

created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),

updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),

UNIQUE (version)

);

CREATE UNIQUE INDEX idx_complaint_tracking_config_one_active

ON complaint_tracking_config ((true)) WHERE is_active = TRUE;

```
```

Seed row, three thresholds treated with three different honesty levels:

- `blocked_resolution_silence_days = 2` — this is **Peter's own real, confirmed decision** (down from an initial 4-day proposal; product doc Sections 3 and 9). Seeded as a real, decided value, `set_by = 'peter@rinconmanagement.com'`, notes citing that resolution — not a placeholder.
- `big_deal_aging_clock_hours = 24` — the product doc leaves this explicitly open ("Exact timing... still open"). This is **Oracle's own proposed default** — one business day feels like the right order of magnitude for "the DO should hear about a stalled big-deal item before it's been sitting a full extra day," but it is a guess, not Peter's decision. Seeded with `set_by = 'system'` and a notes value flagged exactly the way `b2_match_confidence_config`'s own placeholder seed row is flagged: needs Peter's confirmation before it governs a real notification.
- `duplicate_window_days = 3` — also **Oracle's own default**, not specified anywhere in the product doc. "Close together in time" (Section 4's own phrase, contrasted against Category 4 recurrence which is "weeks or months apart") suggests same-day-to-a-few-days; 3 days is a reasonable starting guess, flagged the same way.

Per GOVERNANCE.md Rule 6, changing any of these later is a **Standard** change requiring Peter's approval — a new version row, never an in-place edit.

`complaints.complaint_tracking_config_id` (Section "Proposed Data Model") records which version was in effect for every row, satisfying Rule 5's "every decision must reference the version in effect" — enforced by `complaints_config_required_unless_held`, a real CHECK constraint added by Neo's review (see "Neo's Schema Review" above), not left as a nullable column and a comment.

6. The six categories and the AI categorization model — versioned as code, not as DB config

The six categories themselves (their qualitative definitions — what distinguishes "major money/property risk" from routine spend, for instance) are **not** stored in `complaint_tracking_config`. They're a fixed, structural CHECK enum on `complaints.category` (`legal_compliance`, `blocked_resolution`, `churn_risk`, `escalation_recurrence`, `major_money_property_risk`, `owner_instruction_one_off`) — adding or removing one is realistically a schema/prompt change either way, the same reasoning `maintenance_claims.claim_type` and `operational_notes.access_tier` are plain CHECK enums rather than DB-configurable lists.

What *is* versioned, following the exact

`TERMS_VERSION/TIER_B_CLASSIFIER_VERSION/CATEGORIES_VERSION` precedent already established three times in this codebase (`protected-class-terms.js`, `tier-b-classifier.js`, `component-categories.js`): a plain code constant, `CLASSIFIER_VERSION = 'complaint-categorizer-v1'`, defined in the new categorization module (`complaint-tracking/lib/categorize-complaint.js`), bumped whenever the categorization prompt or logic materially changes. Every AI-categorized complaint's `extracted_by` column stores this string, exactly mirroring `maintenance_claims.extracted_by` — satisfying "every decision must reference the version in effect" without inventing a second config-table mechanism for something that's really a prompt-version question, not a numeric-threshold question.

7. Does this bypass `claims`, or use it?

Bypasses `claims`. A `claims` row is a static, reviewable assertion with a confirm/correct/reject disposition (per `PROPERTY-BRAIN-ARCHITECTURE.md` Section 1.2's own test). A complaint is the opposite of static — it's a live, stateful workflow record with a lifecycle, an owner, delegation, an aging clock, and a duplicate/merge relationship to other rows. Forcing it through `claims`' review-disposition model would be a worse fit than `maintenance_requests` or `security_deposit_cases` (both of which also bypass `claims` for the same reason — a case/ticket with its own lifecycle, not a single fact needing human confirm-or-correct). The AI-extracted parts of a complaint (category, matched subject, tone) are analogous to `maintenance_claims`' own extraction step in *how* they're produced (Design Decisions 2 and 6 reuse that exact machinery), but the record they populate is its own first-class table, not a `claims` row.

8. Subject model, vendor field, and orphan handling — where this deliberately diverges from `operational_notes`

`operational_notes.property_id` is NOT NULL — every operational note is anchored to a property. `complaints.property_id` **is nullable**. This is a deliberate, named divergence, not an oversight: the product doc requires two real cases `operational_notes` never has to handle — a team-member-tied complaint that isn't about any property at all, and a genuinely unmatched orphan complaint with no property, tenant, owner, or vendor identified. Making `property_id` nullable is the honest schema for that.

...

`subject_type` TEXT CHECK (`subject_type` IS NULL OR `subject_type` IN ('owner', 'tenant', 'team_member', 'property'))

`subject_id` UUID -- `owners.id` | `tenants.id` | `team_members.id` | NULL for 'property' or fully unmatched — no enforced FK, same polymorphic pattern `operational_notes` and `audit_log` already use

`needs_matching` BOOLEAN NOT NULL DEFAULT FALSE

...

`needs_matching = TRUE` is a plain, app-set flag (not DB-derived) for the real "couldn't be tied to anything" case the product doc names explicitly — the record still gets created, still shows in the central tool, just flagged for a person to attach by hand later. It is independent of `subject_type` being legitimately 'team_member' with no property at all, which is a *correct* match, not an orphan.

`vendor_id` UUID REFERENCES `vendors(id)` ON DELETE SET NULL — nullable, linked to the real AppFolio-synced vendor list, distinct from `subject_type/subject_id` (who the complaint is *about* vs. which vendor was involved). Populated by a dropdown/autocomplete on manual entry, or the same content-extraction approach used for property/person matching, pointed at vendor names, on the AI path.

9. The "Report an issue" manual-entry form's real access model

Two distinct gates, not one, on the same router:

- `requireActiveTeamMember` — any team member with a real, active `team_members` row (i.e., any real Hub login), regardless of whether they hold any role for `tool='complaint_tracking'` at all. Gates only `POST /api/complaint-tracking/report`.
- `requireComplaintTrackingAccess` — role IN (`'admin'`, `'director_of_operations'`) for `tool='complaint_tracking'`. Gates every other route: the list, the property surfacing, the home count, every stage/review action.

A submission is attributed (`reported_by_team_member_id`, `source = 'manual_staff'`) — never anonymous — and the response is a bare acknowledgment (success + a reference id), never a resource the submitter can query again: no GET route exists that a non-privileged team member's own role could reach to check status. This is enforced by the access split above, not by hiding a route that technically still works.

10. Duplicate detection and merge — a real data model, kept mechanically distinct from recurrence

...

`possible_duplicate_of_id` UUID REFERENCES `complaints(id)` -- self-referential, nullable

`duplicate_status` TEXT NOT NULL DEFAULT 'none' CHECK (`duplicate_status` IN (`'none'`, `'suggested'`, `'confirmed_merged'`, `'dismissed'`))

`merged_into_id` UUID REFERENCES `complaints(id)` -- self-referential, nullable; set only once `duplicate_status = 'confirmed_merged'`

...

On creation, a new complaint is checked against still-open complaints sharing the same (`property_id`, `subject_type`, `subject_id`) triple, created within `duplicate_window_days` (Design Decision 5). A close match sets `duplicate_status = 'suggested'` and populates `possible_duplicate_of_id` — it never auto-merges. A human (DO) confirms (`merged_into_id` set on the newer record, pointing at the primary; the primary keeps all history and stays the live record) or dismisses (`duplicate_status = 'dismissed'`, both stay fully separate, permanently). **A merged-away record is never deleted** — it stays queryable and visible if opened directly, just excluded from active counts (the `complaints_needing_attention` view, Design Decision 14, excludes rows with `merged_into_id IS NOT NULL`).

This is mechanically separate from Category 4 (escalation/recurrence): recurrence is the *same underlying problem* recurring weeks or months apart and must never be merged (merging it would destroy the exact signal recurrence detection needs); a duplicate is the *same event*, close in time, through a different channel. The `duplicate_window_days` threshold (a few days) and recurrence detection (a fast-follow, per the product doc's Section 8, operating over weeks/months) can never collide on the same pair of rows by construction, since recurrence detection is explicitly out of this v1's scope (Section "Open Items").

Not specified here, on purpose: the actual "is this a close match" algorithm (exact subject match is required; how similar the *description* text needs to be, beyond that, is not). Named as a real Open Item for Q, not guessed at.

11. Lifecycle and stage history — audit_log exactly, no parallel table

The product doc's "real, accurate history" requirement (every stage change logged with who/when/why) is satisfied entirely by this codebase's existing audit_log table and writeAuditLog helper (the same one owner-tenant-notes/router.js and maintenance-history/router.js already use) — **no new history table, no bespoke lightweight logging mechanism**. complaints.status holds only the *current* stage; every transition writes one audit_log row.

```
Event	action	actor_type	privacy_category	risk_level
Complaint created (email AI)	complaint_tracking.created	ai_agent	collection	medium
Complaint created (manual)	complaint_tracking.created	human	collection	low
Thread held (Legal/FH placeholder)	complaint_tracking.held	system	collection	high
Protected-class content flagged	complaint_tracking.protected_class_flagged	system (keyword)		
or ai_agent (Tier B / Layer 2)	processing	high — details never carries the flagged text or (absent an		
Asimov extension, Design Decision 2) the matched term				
Stage/status changed	complaint_tracking.stage_changed	human	processing	low, medium if
entering blocked or leaving held — details: { from_status, to_status, reason, actor_role }				
Ownership delegated	complaint_tracking.delegated	human	processing	low
Duplicate suggested	complaint_tracking.duplicate_suggested	system	processing	low
Duplicate confirmed/dismissed	complaint_tracking.duplicate_disposition	human	processing	
low				
Aging nudge computed	complaint_tracking.aging_nudge	system	processing	low
Category 6 note proposed	complaint_tracking.category6_note_proposed	system	collection	
low — cross-reference only; the real proposal event (operational_notes.ai_proposed) is written by				
proposeAINote() itself (Design Decision 13)				
CCPA deletion blocked (held row)	complaint_tracking.ccpa_deletion_blocked_held	system		
processing	medium			
CCPA deletion disposition	complaint_tracking.ccpa_deletion_disposition	human	processing	
medium				
```

All values above (actor_type IN ('human', 'ai_agent', 'system'), privacy_category IN ('collection', 'processing', ...), risk_level IN (... , 'low', 'medium', 'high', ...)) are already legal under audit_log's real, current CHECK constraints (20260815000000_audit_log_rule1_compliance.sql) — confirmed directly, no schema change to audit_log is needed for any event in this table, the same finding the operational_notes migration made for its own new actions.

12. Ownership and the aging/escalation clock

`owner_team_member_id` UUID REFERENCES `team_members(id)` — the accountable owner. For a big-deal complaint, set at creation to whoever currently holds the `director_of_operations` role for `tool='complaint_tracking'` (looked up via `team_member_tool_roles`, never hardcoded to a specific person — if zero or more than one active DO holds that role, the lookup logs an error and the complaint is created with `owner_team_member_id = NULL` plus `needs_human_call = TRUE` rather than guessing).
`delegated_to_team_member_id` UUID REFERENCES `team_members(id)` — nullable; set when the DO hands the actual work to someone else, without changing `owner_team_member_id` (the DO stays accountable, per the product doc's own "owns the process, not necessarily the fix").

`last_aging_nudge_at` TIMESTAMPTZ — nullable; a scheduled job (same manually-triggered-first posture as Design Decision 16 and `email-intake/router.js`'s own precedent) checks every non-resolved big-deal complaint whose time since `created_at` (or its last stage change) exceeds `big_deal_aging_clock_hours`, and hasn't been nudged in the last cycle, and writes `complaint_tracking.aging_nudge`.

Deliberately not designed here: the actual notification channel. The product doc says "the DO gets notified" but never names how (email, an in-Hub badge, an Aircall SMS to the DO's own phone). Building a real send-path is its own integration decision this spec should not invent on Peter's behalf. **v1's concrete, buildable answer:** the aging computation drives an in-Hub visual signal only — a "stalled" badge in the dashboard and a distinct count in the home-page tile — which needs no new integration and satisfies "nothing important should ever be buried." An actual outbound notification (email/SMS) is named as an Open Item requiring Peter's channel decision before Q builds it, not assumed.

13. Category 6 → Operational Notes — calling the real, current `proposeAINote()`

`proposeAINote` (exported from `owner-tenant-notes/router.js`, built and tested tonight against the real database) is an **in-process function, not an HTTP route** — this spec's own router requires it directly (`const { proposeAINote } = require('../owner-tenant-notes/router')`), the same in-process reuse pattern Property 360 already uses for other tools' summary functions. Its real, current required parameters, read from the actual function signature (not assumed):

| Param | Value this build supplies |

|---|---|

| `property_id` (required) | The matched property, if any. **Real gap, inherited from `proposeAINote` itself, not introduced here:** `validateNoteCoreFields` requires a real `property_id` — a Category 6 instruction with no matched property (an owner-level instruction with no single property identified) cannot be proposed as a note at all today. Named as a real, shared Open Item (Section "Open Items"), not solved unilaterally here — solving it means changing `operational_notes.property_id`'s own NOT NULL, which is Neo's and that spec's call, not this one's. |

| `unit_id` | Omitted unless matched. |

| `subject_type` | 'owner' — Category 6 is specifically about owner instructions. |

| `subject_id` | The matched `owners.id`. |

| `note_text` | The categorization model's own drafted note text — for the specific case of an instruction Rincon had to reject as discriminatory, following counsel's own worked template already documented in that spec's

Section 6 ("Owner instruction: ... Rincon response: ..."); for a non-discriminatory one-off instruction, a plain factual statement of the instruction. |

| category | 'owner_instruction' for the general case; 'owner_instruction_rejected' only for the specific discriminatory-rejection case, matching that spec's own existing category value exactly rather than inventing a new one. |

| access_tier | 'management_compliance_restricted' — matches the worked example in that spec's Section 6, given the product doc's own framing that these "occasionally carry real Fair Housing risk." |

| extracted_by (required) | CLASSIFIER_VERSION (Design Decision 6) — this build's own categorization model version string, satisfying proposeAINote's requirement with the one identifier this pipeline already has, no second one invented. |

| routed_to_team_member_id (required) | The same DO lookup as Design Decision 12 — proposeAINote only validates that this is a real, active team_members.id; it does not require any particular role, exactly as that function's own header states. |

| modelFlag / modelCategory | Passed through from the same categorization call's Layer 2 self-report already computed for this complaint's own checkClaim() call (Design Decision 2) — no second AI call. |

On success, complaints.proposed_operational_note_id UUID REFERENCES operational_notes(id) records the created note's id, and a complaint_tracking.category6_note_proposed audit row is written on the complaint's side (Design Decision 11) in addition to proposeAINote's own operational_notes.ai_proposed entry.

The dependency direction is one-way, by design, matching the housing-decision-firewall discipline owner-tenant-notes/router.js **already enforces on itself:** complaint-tracking requires owner-tenant-notes/router.js; nothing in owner-tenant-notes ever requires or references complaint-tracking. operational_notes gains no new column and no FK back to complaints — the one-way pointer above is the only link, and it exists on the calling side only.

14. Tenant lease-end auto-cleanup

Not a stored flag — a query-time view, matching this schema's "computed live" convention already used for call_stats and Property 360's own decision-safe views:

```
```sql
CREATE OR REPLACE VIEW complaints_needing_attention AS
SELECT c.* FROM complaints c
WHERE c.is_big_deal
AND c.status != 'resolved'
AND c.merged_into_id IS NULL
AND (
c.subject_type IS DISTINCT FROM 'tenant'
```

```

OR c.subject_id IS NULL

OR EXISTS (

SELECT 1 FROM leases l WHERE l.tenant_id = c.subject_id AND l.status = 'active'

)

);

'''

```

A tenant-subject complaint drops out of this view automatically once that tenant has no active lease (per `Leases.status`, already synced from AppFolio) — the underlying row is untouched, still queryable directly, never deleted, matching the product doc's "kept on record, never shown as something needing action."

## 15. Access and roles in the Hub

Reuses `team_member_tool_roles` exactly as every other tool does. `tool` gains one new value, `'complaint_tracking'` (the 11th value on that CHECK, following the same DROP-then-ADD pattern this constraint has now used 9 times). **No new role value is needed** — the product doc's access model ("restricted to Peter and the Director of Operations only") maps directly onto the two role values that already exist: `admin` and `director_of_operations`. Unlike `operational_notes`, there is no tier system here at all — access to this tool is binary (you hold one of those two roles for `tool='complaint_tracking'`, or you see nothing beyond the manual "Report an issue" form). This is the simpler of the two access models this codebase now has for sensitive content, and deliberately so: `operational_notes` needed three tiers because it has a broad, portfolio-wide reader population (every property manager); this tool's entire reader population is two people, so a tier system would be real machinery solving a problem that doesn't exist here.

```

```js

const COMPLAINT_TRACKING_ALLOWED_ROLES = ['admin', 'director_of_operations'];

'''

```

Reused, not reinvented: `attachComplaintTrackingRole/requireComplaintTrackingAccess`, the same three-function shape (`attach.../require...Access/require...Role`) every Hub tool already implements independently, evaluated only against rows where `tool='complaint_tracking'` — never inherited from what `admin` or `director_of_operations` mean on any other tool, the same explicit-allow-list discipline `owner-tenant-notes/router.js` documents as a hard-won lesson from the LeadSimple bare-truthy-role bug.

16. Governance path

This is unambiguously a GOVERNANCE.md compliance build under all three of CLAUDE.md's triggers at once: it stores personal data about tenants, owners, and team members; it reads real tenant/owner correspondence to make a triage judgment; and its Category 6 path feeds a system (`operational_notes`) that can influence how staff treat a tenant. Per CLAUDE.md, Asimov and Mason are mandatory gates, not optional ones.

What's already been reviewed, and what hasn't. Asimov and Mason reviewed and cleared the *product-level design* (complaint-tracking-v1-scope.md) — the six categories, the hold/tag split, the tone-signal bias question, the access model. **They have not reviewed this document** — its concrete schema, its two flagged-as-placeholder numeric thresholds (Design Decision 5), the specific `checkClaim()/proposeAINote()` wiring (Design Decisions 2 and 13), or the aging-clock/notification design (Design Decision 12). Per the same discipline the `operational_notes` migration's own header modeled explicitly (never assume a review of one document covers a different, later document it didn't exist to review), this spec recommends a short, focused Asimov/Mason pass on *this* document specifically before Neo/Q build from it — not a re-litigation of the settled product design, just a confirmation that this document's own concrete choices (the placeholder thresholds most of all) don't need adjustment before they're built into a schema.

GOVERNANCE.md Rule 6/7 apply in full to the AI categorization pipeline specifically — it is the first place in this codebase that reads full tenant/owner email content to drive a real human workflow decision, a materially different capability than this week's storage-only Missive connector. Per Rule 7's lifecycle (proposed → risk-assessed → development → shadow → active) and matching `email-intake/router.js`'s own precedent (manually triggered only, no cron, for a 7-day monitored period before Scotty automates it), this pipeline's *categorization* step should get at least the same treatment — arguably its own explicit Asimov sign-off given it is a genuinely new capability (interpretation, not storage), not an extension of the connector Asimov already precheck-approved for storage only.

17. Owner in Distress escalation — live re-verification, not Peter's description taken on faith

The product doc's Section 7 claim ("Did Owner Approve Work?" Yes/No/Undecided; "What is the reason for distress?" Maintenance/Vacancy/Owner Mentioned Selling; churn-risk is the one category that maps to this) was **independently checked live against Rincon's real, live LeadSimple account today (2026-09-09)**, using the real, existing `LEADSIMPLE_API_KEY` already in this environment's `.env` and the same auth/pagination pattern `leadsimple-property-brain/lib/leadsimple-connector.js` already uses. This was not assumed reachable — it was actually queried.

Confirmed live, exactly as Peter described:

- The `Owner in Distress` process type is real and active (`process_type_id`: 966f84bb-f77b-496c-b29b-8071bf5a9cce), with 81 real processes on it as of today (Peter's "80 real cases" is accurate to within one).
- Custom field `did_owner_approve_work` ("Did Owner Approve Work?"), `data_type`: 'choices', options exactly ["Yes", "No", "Undecided"] — confirmed.
- Custom field `what_is_the_reason_for_distress_for_this_owner` ("What is the reason for distress for this owner?"), options exactly ["Maintenance", "Vacancy", "Customer Service", "Owner Mentioned Selling"] — confirmed.

A real gap, found by checking, not by assuming the mapping was complete: the escalation this spec (and the product doc) describes means *creating* a new Owner in Distress process (or setting these fields on one) when a churn-risk complaint fires. LeadSimple's real API surface (`/rest/swagger_doc.json`, checked live) does expose `POST /processes` and `PUT /processes/{process_id}` — a write path exists on LeadSimple's side. But **this codebase's own** `leadsimple-connector.js` is **deliberately, structurally GET-only** ("no

generic request(method, path) helper on purpose — never add a way to pass an HTTP method in from outside this file"), by explicit design choice, not an API limitation. Building the actual write/escalation path is new work this spec does not do — it requires either relaxing that file's own GET-only discipline (a decision for whoever owns that file's design, not this spec) or a new, separately-reviewed write-capable module. Also checked live: the Owner in Distress process type's contact_roles/assignee_roles endpoints both return empty ([]) — meaning there's no configured role template to populate on process creation; its stages are Maintenance / Vacancy / Customer Service / Owner Mentioned Selling / Portfolio Change Needed / Completed / Backlog, which line up with the reason-for-distress values, suggesting the natural creation shape (create at the stage matching the churn reason) but this is inferred from the live stage list, not confirmed against a real create-a-process example.

Named, unresolved verification item, stated honestly rather than assumed away: the read-side field mapping is now independently confirmed live. The write-side mechanism (how a new Owner in Distress process actually gets created from this pipeline, what fields are required to create one successfully, whether the empty contact_roles/assignee_roles matter) is not built and not verified beyond confirming the raw endpoints exist — Q needs to do a real test-create against a non-production-consequential record (or with Peter's sign-off on a real one) before this escalation path ships.

The Ingestion Pipeline — Concretely, Against Real Tables

New module: complaint-tracking/lib/process-pending-messages.js, a manually-triggered route (POST /api/complaint-tracking/process-pending, x-cron-secret-gated, no automatic schedule yet — same posture as email-intake/router.js's own ingestion route, per Design Decision 16):

1. Query missive_message_intake WHERE pipeline_status = 'pending', group by missive_conversation_id.
2. For each conversation: adapt into checkThread's thread shape (Design Decision 1) and call it FIRST — before any property/tenant/owner matching of any kind. **Corrected by Neo's review (see "Neo's Schema Review" above and Design Decision 3):** the original draft ran the Layer 0 shallow match before this step, on every thread; that inverted missive_message_intake's own governance-cleared ordering requirement (property/tenant/owner association only after the hold check has run) and is fixed here.
3. **Held:** run the Layer 0 shallow match now, only now (Design Decision 3), to populate the placeholder's subject; insert one complaints row (held_legal_fair_housing = TRUE, status = 'held', subject from Layer 0); mark every message in the conversation pipeline_status = 'processed'; write complaint_tracking.held. Stop — no categorization, no content check, ever, for this thread.
4. **Not held:** Layer 0 does not run at all for this thread — its property/subject match comes entirely from categorization's own extraction, below. Run categorization (complaint-tracking/lib/categorize-complaint.js, CLASSIFIER_VERSION, Design Decision 6) against the full thread text, producing category (nullable), needs_human_call, tone_trend, a summary, extracted property/subject/vendor names (resolved against real rows, the same "AI extracts text, a separate step resolves it to a UUID" pattern extract-claims.js already uses — not designed fresh here), and the Layer 2 self-report (modelFlag/modelCategory).
5. Run checkClaim() (Design Decision 2) against the full thread text with that self-report.

6. Run duplicate detection (Design Decision 10).
7. Insert the complaints row; if category = 'owner_instruction_one_off', also call proposeAINote() (Design Decision 13).
8. Write complaint_tracking.created (and .protected_class_flagged / .duplicate_suggested if applicable).
9. Mark every message in the conversation pipeline_status = 'processed'.

No FK is added from missive_message_intake to complaints in either direction (that table's own migration is explicit that it carries zero FKs, by design). Traceability is one plain TEXT column: complaints.source_missive_conversation_id, the same "external system, no enforced FK" convention already used for call_stats.aircall_user_id.

Proposed Data Model (for Neo to finalize)

```
```sql
CREATE TABLE IF NOT EXISTS complaint_tracking_config (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 version INTEGER NOT NULL,
 blocked_resolution_silence_days INTEGER NOT NULL CHECK (blocked_resolution_silence_days > 0),
 big_deal_aging_clock_hours INTEGER NOT NULL CHECK (big_deal_aging_clock_hours > 0),
 duplicate_window_days INTEGER NOT NULL CHECK (duplicate_window_days > 0),
 is_active BOOLEAN NOT NULL DEFAULT TRUE,
 set_by TEXT NOT NULL,
 set_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
 notes TEXT,
 created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
 updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
 UNIQUE (version)
);

ALTER TABLE complaint_tracking_config ENABLE ROW LEVEL SECURITY;

CREATE UNIQUE INDEX idx_complaint_tracking_config_one_active
ON complaint_tracking_config ((true)) WHERE is_active = TRUE;

-- Seed version 1 — see Design Decision 5 for exactly which of these three
```

-- values is Peter's real decision vs. Oracle's flagged placeholder.

INSERT INTO complaint\_tracking\_config

(version, blocked\_resolution\_silence\_days, big\_deal\_aging\_clock\_hours, duplicate\_window\_days, is\_active, set\_by, notes)

VALUES (

1, 2, 24, 3, TRUE, 'peter@rinconmanagement.com',

'blocked\_resolution\_silence\_days=2 is Peter's real, confirmed decision (down from an initial 4-day proposal — complaint-tracking-v1-scope.md Sections 3 and 9). big\_deal\_aging\_clock\_hours=24 and duplicate\_window\_days=3 are Oracle's own reasonable defaults, NOT yet confirmed by Peter — flagged here exactly like b2\_match\_confidence\_config's own placeholder seed row. Confirm both before this governs a real DO notification or a real merge decision.'

);

CREATE TABLE IF NOT EXISTS complaints (

id UUID PRIMARY KEY DEFAULT gen\_random\_uuid(),

-- Subject / location — nullable throughout; see Design Decision 8 for

-- why this deliberately diverges from operational\_notes.property\_id's

-- NOT NULL.

property\_id UUID REFERENCES properties(id) ON DELETE RESTRICT,

unit\_id UUID REFERENCES units(id) ON DELETE SET NULL,

vendor\_id UUID REFERENCES vendors(id) ON DELETE SET NULL,

subject\_type TEXT CHECK (subject\_type IS NULL OR subject\_type IN ('owner', 'tenant', 'team\_member', 'property')),

subject\_id UUID, -- owners.id | tenants.id | team\_members.id | NULL — no enforced FK, same polymorphic pattern as operational\_notes/audit\_log

needs\_matching BOOLEAN NOT NULL DEFAULT FALSE,

-- Category / significance (product doc Sections 2-3)

category TEXT CHECK (category IS NULL OR category IN (

'legal\_compliance', 'blocked\_resolution', 'churn\_risk',

'escalation\_recurrence', 'major\_money\_property\_risk', 'owner\_instruction\_one\_off'

)),

needs\_human\_call BOOLEAN NOT NULL DEFAULT FALSE,

held\_legal\_fair\_housing BOOLEAN NOT NULL DEFAULT FALSE,

description TEXT, -- AI summary or the manual reporter's own text; NULL only for held placeholders (enforced below)

-- Blocked-resolution detail (Design Decision 4)

blocked\_reason TEXT CHECK (blocked\_reason IS NULL OR blocked\_reason IN ('explicit\_refusal', 'inferred\_from\_silence')),

blocked\_party TEXT CHECK (blocked\_party IS NULL OR blocked\_party IN ('owner', 'tenant')),

blocked\_since TIMESTAMPTZ,

-- Fair Housing content tag (Design Decision 2) — advisory metadata only,

-- no review/visibility-exclusion workflow needed here (unlike

-- operational\_notes) because this tool's entire reader population

-- (Peter + DO) already sees everything.

flagged\_protected\_class BOOLEAN NOT NULL DEFAULT FALSE,

flagged\_category TEXT,

-- Frustration/tone (product doc Section 3a) — advisory only, never

-- triggers any action on its own.

tone\_trend TEXT CHECK (tone\_trend IS NULL OR tone\_trend IN ('stable', 'escalating')),

-- Lifecycle (product doc Section 4)

status TEXT NOT NULL DEFAULT 'open' CHECK (status IN ('open', 'in\_progress', 'blocked', 'resolved', 'held')),

resolution\_note TEXT, -- required once status = 'resolved' (enforced below)

-- Ownership (Design Decision 12)

owner\_team\_member\_id UUID REFERENCES team\_members(id),

delegated\_to\_team\_member\_id UUID REFERENCES team\_members(id),

last\_aging\_nudge\_at TIMESTAMPTZ,

-- Provenance

source TEXT NOT NULL CHECK (source IN ('email\_ai', 'manual\_staff')),

reported\_by\_team\_member\_id UUID REFERENCES team\_members(id), -- required for manual\_staff

extracted\_by TEXT, -- CLASSIFIER\_VERSION string, required for email\_ai UNLESS held (Design Decision 6; a held row is source='email\_ai' but was never AI-categorized, so it has no classifier version to record — see complaints\_source\_requires\_fields below, fixed by Neo's review)

source\_missive\_conversation\_id TEXT, -- plain text, no FK — traceability back to missive\_message\_intake (external system, sync-order not guaranteed, same convention as call\_stats.aircall\_user\_id)

complaint\_tracking\_config\_id UUID REFERENCES complaint\_tracking\_config(id), -- which threshold version was in effect (Rule 5); required on every non-held row — see complaints\_config\_required\_unless\_held below, added by Neo's review

-- Duplicate detection (Design Decision 10)

possible\_duplicate\_of\_id UUID REFERENCES complaints(id),

duplicate\_status TEXT NOT NULL DEFAULT 'none' CHECK (duplicate\_status IN ('none', 'suggested', 'confirmed\_merged', 'dismissed')),

merged\_into\_id UUID REFERENCES complaints(id),

-- Category 6 -> Operational Notes (Design Decision 13) — one-way

-- pointer only; operational\_notes has no reciprocal reference.

proposed\_operational\_note\_id UUID REFERENCES operational\_notes(id),

created\_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),

updated\_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),

-- "Big deal" is exactly: a confirmed category, genuine AI uncertainty,

-- or a held item — computed once, stored, so every query site (list,

-- home count, Property 360, the aging job) agrees by construction

-- rather than re-deriving this logic in N places.

is\_big\_deal BOOLEAN GENERATED ALWAYS AS (

category IS NOT NULL OR needs\_human\_call OR held\_legal\_fair\_housing

) STORED,

CONSTRAINT complaints\_held\_excludes\_ai\_fields CHECK (

held\_legal\_fair\_housing = FALSE OR (

category IS NULL AND needs\_human\_call = FALSE AND description IS NULL

AND flagged\_protected\_class = FALSE AND tone\_trend IS NULL

)

),

CONSTRAINT complaints\_flag\_requires\_category CHECK (

flagged\_protected\_class = FALSE OR flagged\_category IS NOT NULL

```
),
CONSTRAINT complaints_blocked_requires_reason CHECK (
category IS DISTINCT FROM 'blocked_resolution' OR blocked_reason IS NOT NULL
),
```

```
CONSTRAINT complaints_resolved_requires_note CHECK (
status IS DISTINCT FROM 'resolved' OR resolution_note IS NOT NULL
),
```

```
-- FIXED by Neo's review (see "Neo's Schema Review" above): the original
-- version of this constraint required extracted_by IS NOT NULL for every
-- source='email_ai' row with no exception for held rows — but a held row
-- (held_legal_fair_housing = TRUE) is inserted by the email pipeline
-- (source='email_ai') and is, by design (complaints_held_excludes_ai_fields
-- above), never AI-categorized, so it genuinely has no classifier version
-- to record. As originally written, no held row could ever satisfy both
-- constraints at once — this exempts held rows from the extracted_by
-- requirement instead of inventing a fake version string to satisfy a
-- constraint that shouldn't apply to them.
```

```
CONSTRAINT complaints_source_requires_fields CHECK (
(source = 'manual_staff' AND reported_by_team_member_id IS NOT NULL AND extracted_by IS NULL)
OR (source = 'email_ai' AND held_legal_fair_housing = TRUE AND extracted_by IS NULL)
OR (source = 'email_ai' AND held_legal_fair_housing = FALSE AND extracted_by IS NOT NULL)
),
```

```
CONSTRAINT complaints_merge_requires_confirmed CHECK (
merged_into_id IS NULL OR duplicate_status = 'confirmed_merged'
),
```

```
-- ADDED by Neo's review (see "Neo's Schema Review" above): GOVERNANCE.md
-- Rule 5 requires every decision that used a versioned threshold to
-- reference the version in effect. Duplicate detection (Design Decision
-- 10) runs against complaint_tracking_config's duplicate_window_days for
-- every non-held complaint at creation — manual or email_ai alike — so
-- every non-held row genuinely used a config version and must record
```

```
-- which one, not just be allowed to. A held row uses no threshold at all
-- (it skips categorization, content-check, and duplicate detection
-- entirely — Ingestion Pipeline step 3) and so correctly has no config
-- version to reference.
```

```
CONSTRAINT complaints_config_required_unless_held CHECK (
held_legal_fair_housing = TRUE OR complaint_tracking_config_id IS NOT NULL
)
);
```

```
ALTER TABLE complaints ENABLE ROW LEVEL SECURITY;
```

```
CREATE INDEX idx_complaints_property ON complaints(property_id) WHERE property_id IS NOT NULL;
```

```
CREATE INDEX idx_complaints_subject ON complaints(subject_type, subject_id) WHERE subject_id IS NOT
NULL;
```

```
CREATE INDEX idx_complaints_needs_matching ON complaints(needs_matching) WHERE needs_matching =
TRUE;
```

```
CREATE INDEX idx_complaints_duplicate_suggested ON complaints(duplicate_status) WHERE duplicate_status
= 'suggested';
```

```
CREATE INDEX idx_complaints_big_deal_open ON complaints(status) WHERE is_big_deal AND status !=
'resolved';
```

```
CREATE INDEX idx_complaints_aging_candidates ON complaints(created_at) WHERE is_big_deal AND status
NOT IN ('resolved');
```

```
DROP TRIGGER IF EXISTS trg_complaints_updated_at ON complaints;
```

```
CREATE TRIGGER trg_complaints_updated_at
```

```
BEFORE UPDATE ON complaints FOR EACH ROW EXECUTE FUNCTION set_updated_at();
```

```
-- Design Decision 14 — tenant lease-end auto-cleanup, computed live.
```

```
CREATE OR REPLACE VIEW complaints_needing_attention AS
```

```
SELECT c.* FROM complaints c
```

```
WHERE c.is_big_deal
```

```
AND c.status != 'resolved'
```

```
AND c.merged_into_id IS NULL
```

```
AND (
```

```
c.subject_type IS DISTINCT FROM 'tenant'
```

OR c.subject\_id IS NULL

OR EXISTS (SELECT 1 FROM leases l WHERE l.tenant\_id = c.subject\_id AND l.status = 'active')

);

...

audit\_log — **no schema change**. Every event in Design Decision 11's table uses actor\_type/privacy\_category/risk\_level values already legal under audit\_log's real, current CHECK constraints — confirmed directly, same finding the operational\_notes migration made for its own new actions.

## Access / Roles in the Hub

team\_member\_tool\_roles.tool gains one new value: 'complaint\_tracking' (11th value, same DROP-then-ADD pattern used 9 times already). No role value is added — admin and director\_of\_operations already exist and are the only two roles this tool ever checks (Design Decision 15). No seed/grant row is inserted by the migration; who actually holds these roles for this tool is Peter's call, made after Q has built something to grant access to, same deferral every prior tool onboarding has used.

## Data Inventory (GOVERNANCE.md Rule 4)

- pii\_fields: description (highest density — complaint content, by design); resolution\_note; blocked\_reason/blocked\_party/blocked\_since (identify who is refusing/being unresponsive); flagged\_category (could indirectly reveal the sensitive topic without containing it); subject\_id/subject\_type (identifies a specific tenant/owner/team member); source\_missive\_conversation\_id (traces back to real correspondence).
- agents\_with\_access: the categorization pipeline (Claude, via ANTHROPIC\_API\_KEY — already resolved for this exact use, per operational\_notes-SPEC.md Section 8's confirmed DPA/Commercial-Terms answer, which applies identically here since it's the same direct-API credential); Hub users holding admin or director\_of\_operations for tool='complaint\_tracking'; any active team member, for the manual-report form only (write access to their own submission, no read access to anything).
- privacy\_category: Complaint/dispute record about a tenant, owner, or team member — a new category for this schema (closest existing analog is operational\_notes' "owner/tenant operational record," but this one also covers team-member-tied complaints, which that table never does).
- retention\_policy: **RESOLVED, not a placeholder** — the product doc's own explicit "never delete" stance (Section 4) is a real, stated retention decision, not an unaddressed question: indefinite retention, matching the pattern already used elsewhere in this schema (e.g. call\_stats' own resolved indefinite retention). Distinct from deletion policy — see ccpa\_deletable below.
- ccpa\_exportable: TRUE — this table plainly holds personal data about identifiable tenants, owners, and team members.

- `ccpa_deletable`: TRUE for ordinary rows, via the same targeted `description/resolution_note` → "[REDACTED]" redaction convention `operational_notes.note_text` already uses, preserving `subject_type`, `category`, `status`, and dates for audit continuity.

`held_legal_fair_housing = TRUE` rows are a categorical exception — this tool's own equivalent of `operational_notes`' `legal_privileged` carve-out, designed the same way and for the same reason. A `held` placeholder represents (or is created because of a mechanical match for) an actual formal Fair Housing/HUD/CRD complaint or real attorney/legal correspondence — exactly the spoliation/litigation-hold risk `operational_notes`' Section 4 already reasoned through for its own `legal_privileged` tier, even though this table has no tier system at all otherwise. The same mechanism applies: the standard redaction path must **hard-refuse** on any `held_legal_fair_housing = TRUE` row (never a silent skip), routing instead to a fresh, per-request hold/exception determination (no cached "legal hold" column — a hold attaches to a matter, checked at request time). Given this tool's access model is already just `admin/director_of_operations` (Design Decision 15), the determination is made by whichever of those two roles handles the request — no separate named-attorney-confirmation gate is invented here, mirroring Peter's own already-made decision on the identical question for `operational_notes` (an internal-admin call, not requiring outside counsel sign-off per request). Two audit actions, reused by name pattern from `operational_notes`: `complaint_tracking.ccpa_deletion_blocked_held` (system, automatic, fires every time the redaction endpoint is hit for a held row) and `complaint_tracking.ccpa_deletion_disposition` (human, the actual determination — details captures the reasoning, never the held row's own content, which in practice is minimal or empty anyway per Design Decision 1).

- **RLS**: enabled on both new tables, zero permissive policies at creation — matches every table in this schema; every reader today connects via the service-role key, which bypasses RLS regardless.
- **Rule 10** (`handleCCPADelete`): for a tenant/owner contact-deletion request, identify every `complaints` row where `subject_id = contact_id` (manual lookup via `subject_id`, same accepted v1 limitation `operational_notes` already carries — not automatic), apply the redaction path above (with the `held_legal_fair_housing` carve-out honored), confirm completion, log it permanently.

## Routes Needed (for Q)

- POST `/api/complaint-tracking/report` — manual submission (`requireActiveTeamMember` only).
- POST `/api/complaint-tracking/process-pending` — the ingestion pipeline (Section "The Ingestion Pipeline"), `x-cron-secret-gated`, manually triggered only (Design Decision 16).
- GET `/api/complaint-tracking` — full filtered list (`requireComplaintTrackingAccess`), returning `is_big_deal` per row so the dashboard splits big-deal-up-top vs. routine-behind-a-dropdown without re-deriving that logic client-side.
- GET `/api/complaint-tracking/home-count` — the Hub home-page tile count.
- GET `/api/complaint-tracking/property/:property_id` — Property 360 surfacing, big-deal items only, full stop (queries `complaints_needing_attention` filtered further to `property_id`).
- POST `/api/complaint-tracking/:id/stage` — status transitions, with a required note for anything other than a straight Open → In Progress move (Design Decision 11).

- POST /api/complaint-tracking/:id/delegate — sets delegated\_to\_team\_member\_id.
- POST /api/complaint-tracking/:id/match — attaches an orphan (needs\_matching = TRUE) record to a real property/subject/vendor by hand.
- POST /api/complaint-tracking/:id/duplicate/confirm and .../dismiss — the merge disposition (Design Decision 10).
- POST /api/complaint-tracking/:id/held/close — the closure-note flow for a held placeholder (sets resolution\_note, status = 'resolved').
- POST /api/complaint-tracking/:id/redact — the CCPA path, with the held\_legal\_fair\_housing hard-refuse-and-disposition flow above.

## Open Items — Needs Confirming Before This Gets Built

1. big\_deal\_aging\_clock\_hours = 24 **and** duplicate\_window\_days = 3 are Oracle's own defaults, not Peter's decisions — confirm or adjust before this seed row governs a real DO notification or a real merge (Design Decision 5).
2. **The actual notification channel for a stalled big-deal item** (email, in-Hub badge, Aircall SMS) is not specified by the product doc and not designed here — v1 ships with an in-Hub visual signal only; an outbound channel is a separate decision (Design Decision 12).
3. **Layer 3's real trigger** — how a staff member's "Legal Hold — do not auto-process" tag in Missive actually reaches this pipeline — has no built data source yet; legalHoldTag defaults false until this exists (Design Decision 1).
4. **The duplicate "close match" algorithm** beyond exact-subject-match is not specified (Design Decision 10) — a real implementation choice for Q, not guessed at here.
5. proposeAINote's property\_id NOT NULL **requirement** has no answer today for an owner-level Category 6 instruction with no single matched property (Design Decision 13) — a real, shared gap with operational\_notes itself, not solved unilaterally by this spec.
6. **The Owner in Distress write path** (creating/updating a real LeadSimple process) is unbuilt and only partially verified live — the read-side field mapping is confirmed; the create-a-process mechanics are not (Design Decision 17).
7. **A short, focused Asimov/Mason pass on this document specifically** — not the already-cleared product design — covering the two placeholder thresholds, the checkClaim()/proposeAINote() wiring, and the aging-clock design (Design Decision 16).
8. **Whether a dedicated team-member profile page is ever wanted** for team-member-tied complaints — the product doc already declined this for v1 (its own central-tool filter is enough); restated here only so it isn't mistaken for an oversight in this spec.
9. **Added by Neo's review: confirm the corrected Layer 0 ordering (Design Decision 3) with Asimov specifically, not just as part of Item 7's general pass.** This document's original draft ran the shallow property/tenant/owner address-match on every thread before the hold check — a real conflict with missive\_message\_intake's own governance-cleared Rule 9 Housing-Decision Firewall, which requires that association to happen only after the hold check has run. This spec now runs Layer 0 only after a thread is

confirmed held. Asimov should confirm this correction is sufficient before Q builds the ingestion pipeline — this is a conflict with a specific prior compliance decision on the exact table this pipeline reads from, not a new judgment call being made for the first time.

## Rough Build Size

Larger than `call_stats` (no AI, no content check there) but smaller than `operational_notes`' full build (no tier system, no portfolio-wide access problem, no acknowledgment-gate machinery). Roughly: **Neo ~1–2 sessions** (two new tables, one view, one CHECK-constraint widening, no new role value). **Q ~4–6 sessions** (the ingestion adapter and pipeline; the categorization model call; wiring `checkThread/checkClaim/proposeAINote`; duplicate detection; the aging job; the manual-report route; the CCPA/held redaction flow) — the largest single piece of new work in this codebase's history to touch real tenant/owner email content for a triage decision, and should be estimated and reviewed accordingly, not compressed to match a smaller tool's timeline. **Tron ~1–2 sessions** (the big-deal/dropdown list view, the Property 360 flat-surfaced section, the home-page tile, the owner/tenant visual-separation requirement from the product doc's Section 4). Asimov/Mason: one short pass per Open Item 7 above, not a full second design review.